

# Automated testing as an aid to systems integration

I D Hicks, G J South and A O Oshisanwo

---

*Within BT's Systems Integration department, the integration and testing of large, complex software systems often requires large and complex test regimes. Optimising the activities and time-scales within this part of the life cycle will invariably involve the application of test tools. This paper sets out the main elements required for successful test automation. It begins by introducing test complexity measures in process testing of a character-based user interface software system. This is then developed to describe a theoretical approach to the application of test automation in the context of graphical user interface (GUI) software systems and discusses methodologies for test-case design and reuse, to achieve the maximum benefit of tool utilisation. The paper concludes with a description of the successful application of test automation within two major systems integration projects, Work Manager and MCSS for Cambridge.*

---

## 1. Introduction

One impetus for automating testing is the need to reduce the costs associated with testing [1] and, if applied correctly, this can increase the amount of testing in a given time or, alternatively, reduce the time required to test. This can create more time for the design of new tests leading to increased test coverage and ultimately improvements in the quality of the delivered system.

As software systems become larger and more complex, the ability to find the resource to comprehensively test and verify the operation of them becomes virtually impossible. Increasingly, the time-scales for delivery of these systems are perceived to be inversely related to their complexity and this, coupled with the need to comprehensively test, will require increased usage of tools and techniques in order to realise the timely delivery of these systems.

Automating one or many aspects of software testing must produce benefits if it is to be of value. Automation should provide benefits not easily realisable when manually testing. Potential benefits are:

- it can replace labour intensive manual testing,
- it can provide improved testing repeatability and accuracy,
- it has the potential to reduce costs.

With regard to this last point, a UK report commissioned by the Department of Trade and Industry [2] suggests that the application of software testing tools would result in an overall saving of 9–18% for system development.

While there are significant advantages in automating testing, there are other considerations that must be addressed if successful test automation is to be achieved. These considerations may be summarised as:

- the experimental approach to testing is lost,
- initial costs may be high,
- there are risks involved with automating an evolving system.

Systems Integration (SI) has applied techniques to realise the benefits of test automation while addressing the considerations noted above. The techniques were developed as part of MSc thesis work [3, 4] and it is a summary of these techniques that are presented in this paper, together with specific examples of their application.

## 2. Test complexity measures

Current techniques for quantifying the benefits of software test automation are in their infancy. Most techniques do not fully consider the complexity of the software in terms of its testability and whether this ever becomes too great to provide any benefits from the automation. This section presents an approach to the analysis of complexity that was developed as part of an MSc thesis [3] and utilised during the Work Manager integration and test activities.

### 2.1 Complexity in testability

Test complexity measures are a way of trying to understand how the complexity of the activities during the testing process impact on the potential benefits of test automation and whether test automation would yield any cost savings. Intuitively, this relationship between test cost saving and test complexity may be reasoned as shown in Fig 1. Understanding whether, and in what way, the complexity of testing reaches point A, will be of interest to any test manager.

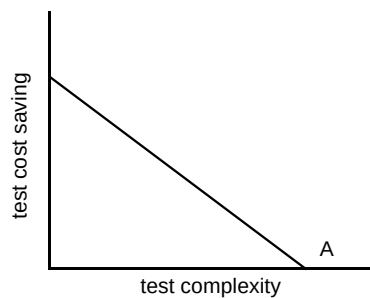


Fig 1 Intuitive relationship between test cost savings and test complexity.

Complexity needs to be understood in terms of software testability, not in terms of the software construction itself, although it may be a factor.

To better define test complexity we need to understand the testability features and how to apportion the time of the test automation activity. This is discussed in the following section.

### 2.2 Testability features

Testability features are derived from the activities required to execute a test case. These need to cover all aspects of the input/output specifications of the IEEE 829 Test Documentation standard [5]. These may be directly dependent on the test environment, but an attempt to generalise them is presented in the following list:

- test database,
- files and logs,
- interfacing user transaction,
- real-time evaluations, secondary test tools,
- single-shot test stimuli,
- periodic/alarmed test stimuli,
- operating system.

### 2.3 Time metric categories

The manpower cost of applying test automation to a test suite can be understood in terms of time metric categories. These have been derived for test automation activities as described in Table 1.

Table 1 Time metric categories.

| ID | Time metric category                                                   |
|----|------------------------------------------------------------------------|
| 1  | Time spent on the test automation design                               |
| 2  | Time spent implementing test automation                                |
| 3  | Time spent correcting failed test automation execution                 |
| 4  | Time spent analysing test results and correcting or tuning test values |
| 5  | Time spent executing tests in a test phase                             |

The examples in section 2.5 will illustrate how and why these metric categories are used.

### 2.4 Test complexity measures

Test complexity measurement can be categorised as static and dynamic.

**Static analysis** is measuring the average occurrence of each testability feature per test case. This enables the overall levels of testability features within the test activity to be quantified, hence identifying the main test activities.

This could be useful for identifying the requirements for certain skills (e.g. database skills) or test automation approaches (e.g. tool selection).

**Dynamic analysis** is the measurement of the time spent on each testability feature for every time metric category. This provides a view of how the test activity time was spent across the testability features, hence identifying the main effort areas. This is useful for suggesting target areas for test process improvement (e.g. improve interface to secondary test tools).

## 2.5 Examples

To illustrate dynamic and static complexity measures, three sub-systems of the BT Work Manager (WM) product were subject to this analysis. In order of perceived complexity, sub-system 1 (SS1) was a database processing program, run as a single-shot executable, sub-system 2 (SS2) was a set of external interface management processes and sub-system 3 (SS3) was a real-time work allocation algorithm. The highlights of the detailed work [3] are briefly described in Table 2, Table 3, and Table 4. The results of the static analysis are shown in Table 2.

Table 2 Average occurrence of a testability feature/test case.

|   | Testability feature           | SS1  | SS2  | SS3   |
|---|-------------------------------|------|------|-------|
| 1 | Test database                 | 3.13 | 1.69 | 10.08 |
| 2 | Files and logs                | 0.02 | 6.02 | 0.21  |
| 3 | Interfacing user transactions | 0    | 0    | 1.92  |
| 4 | Real-time evaluations         | 0.09 | 0.03 | 1.49  |
| 5 | Secondary test tools          | 0    | 1.76 | 3.19  |
| 6 | Single-shot test stimuli      | 1    | 0    | 1.83  |
| 7 | Periodic/alarmed test stimuli | 0    | 1.64 | 0.09  |
| 8 | Operating system              | 0.01 | 0.2  | 0.18  |

The dynamic analysis was performed on SS3 only. This analysis was limited to time metric category 2 (time spent implementing test automation code) and time metric categories 3/4 combined (time spent trying to get the test script to work). The results are given in Table 3.

Table 3 Total time (hours) spent in time metric category 2 and 3/4 for each testability feature (SS3 only).

|   | Testability feature           | Time metric Cat 2 | Time metric Cat 3/4 |
|---|-------------------------------|-------------------|---------------------|
| 1 | Test database                 | 17.2              | 14                  |
| 2 | Files and logs                | 1                 | 1.6                 |
| 3 | Interfacing user transactions | 5.15              | 3.2                 |
| 4 | Real-time evaluations         | 5.6               | 5                   |
| 5 | Secondary test tools          | 12.75             | 8.6                 |
| 6 | Single-shot test stimuli      | 4.3               | 2.7                 |
| 7 | Periodic/alarmed test stimuli | 0                 | 0.2                 |
| 8 | Operating system              | 1                 | 0.2                 |

From initial observation, it appears that SS3 had most time spent on implementing the automation where there was database activity. This, however, needs to be seen with respect to the volume of that feature. The average values, given in Table 4, are derived from Table 2 and Table 3 and are based on 106 test cases for SS3. Testability features 2, 7 and 8 have occurred less than once per test case; hence they are likely to be too sensitive to give a reliable average.

Table 4 highlights the testability features that were most expensive in terms of time spent on them. These were real-time evaluations, followed by the use of secondary test tools. Interestingly, the test database activities appear to require the least amount of effort to automate.

Table 4 Average time (min) spent in time metric category.

|   | Testability feature           | Time metric cat 2 | Time metric cat 3/4 |
|---|-------------------------------|-------------------|---------------------|
| 1 | Test database                 | 0.97              | 1.75                |
| 2 | Files and logs                | 2.7               | 7.01                |
| 3 | Interfacing user transactions | 1.52              | 2.46                |
| 4 | Real-time evaluations         | 2.13              | 4.03                |
| 5 | Secondary test tools          | 2.26              | 3.79                |
| 6 | Single-shot test stimuli      | 1.33              | 2.17                |
| 7 | Periodic/alarmed test stimuli | 0                 | 1.26                |
| 8 | Operating system              | 3.14              | 3.77                |

While this analysis would benefit from a more extensive range of test scenarios to be useful, it does indicate that certain test activities appear to require significantly greater test automation effort.

## 3. A coherent approach to test automation

This section introduces a general approach to test automation, with the emphasis on reuse and integration into integrated verification, validation and testing (IVV&T) practices.

### 3.1 GUI software systems testing

The use of GUI-based test tools introduces a further element of complexity into the testing arena. These tools, which are generally of the capture/replay type, not only test the functionality of the application but are required to perform this task in a graphical environment.

One of the reasons why testing within this environment is made more difficult than conventional character-based environments is due to the many changes the user interface can undergo early in the development cycle. This could be due to changing requirements or fixes to reported faults, or simply new industry trends in GUI design. These changes require the maintenance of test scripts to address both the changed functionality and the altered interface.

It has been reported [6] that GUI-oriented testing can be as difficult as developing the application under test and that as a result software testers are required to become test code developers in order to fully utilise the test script aspect of GUI test tools. Understanding the problems associated with GUI testing can help in implementing successful automation and there are many tools available now which are using new techniques aimed specifically at addressing

some of these known problems. One such example, which addresses changes in the GUI, is the context-sensitive tool that uses the semantic information and not the positional information when replaying a previously recorded test script.

### 3.2 Tool utilisation

However, application of technology alone will not produce the benefits identified in section 1. While using formally evaluated tools is fundamental to the success of automating, the main benefits will only be achieved if the tools are used in tandem with appropriate working practices and processes. The result of not doing this is low tool utilisation. A tool utilisation survey carried out during the production of the Computer Aided Software Testing (CAST) report [7] highlighted some of the reasons attributable to this trend, the most significant factor being implementation of the tool within the organisation. Beizer [8] and others argue that low usage is attributable to:

- training deficiencies,
- expectations concerning tool performance,
- reluctance to change the current working practices,
- ignorance of the potential benefits of tools,
- lack of a practical automation strategy.

These issues can and often do lead to the tool becoming 'shelfware'. Integrating tool use within traditional VV&T operations is discussed in more detail later and seeks to address some of the issues discussed above.

### 3.3 A structured approach

A common approach within the industry to test automation is to apply a subjective measure relating to how 'boring' a test case is to execute. These test cases are then selected for automation. The measure can be represented graphically (see Fig 2).

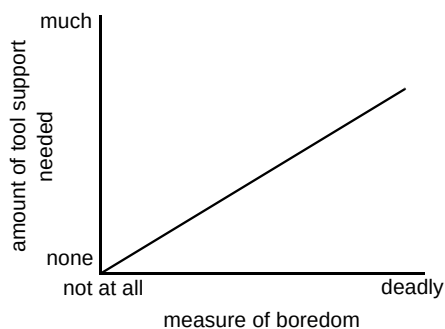


Fig 2 Boredom index (source — CAST report [7]).

Within SI this has been developed into an objective method for selecting test cases to automate. This has involved redefining the meaning of the word 'boring'. The conclusion is that test cases that are perceived to be boring are in fact those exhibiting the greatest simplicity. This then allows test cases to be selected on the basis of complexity measures, as discussed earlier.

While this is an improvement on previous schemes, enhancements to the overall selection process can be made.

### 3.4 Automation framework

Introducing a framework for automation means that techniques such as complexity measures can be applied to a more general scheme that takes into consideration other factors, such as reuse. The framework also ensures the issues of manual and automatic test coexistence are addressed and it achieves this through the use of a formal procedure for test case selection. This is discussed in more detail in the following section.

### 3.5 The selection process

The earlier ideas of complexity measures are applied to test-case selection together with new measures. These new measures include reuse and frequency of reuse. The measures are applied within a well-defined procedure that results in groupings of candidate test cases. These groupings are hierarchically arranged, the 'top' grouping representing the most favoured tests to automate and the 'bottom' the least favoured.

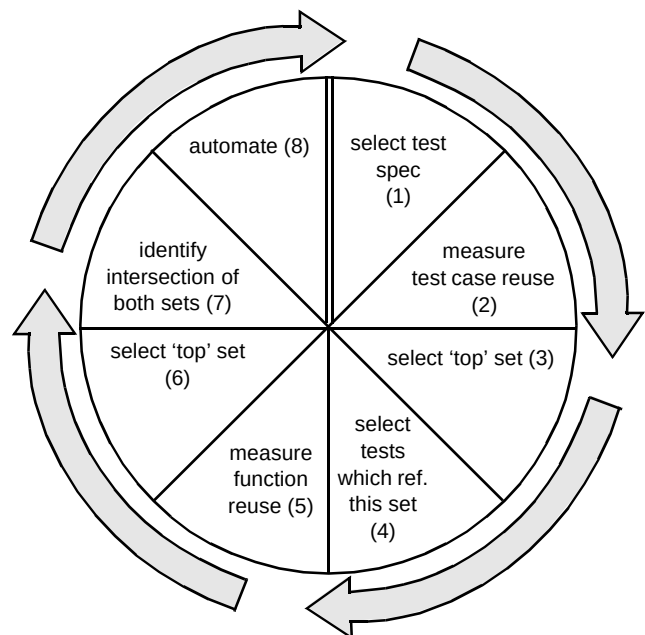


Fig 3 Test case selection process.

Figure 3 shows the process for selecting test cases to automate. It indicates that the process repeats itself, the number of iterations being sensitive to:

- the stability of the GUI,
- the number of test cases in a test specification.

Figure 4 provides an example of the test case selection procedure in action.

- (1) Select a test specification. This will contain detailed descriptions of each test case.
- (2) Analyse the specification for test cases that are reused and note down this frequency of occurrence. The frequency is identified as '(x3)', '(x6)', for example.
- (3) Identify the dominant test cases, i.e. those exhibiting a high frequency count. This constitutes Set A.
- (4) Analyse the specification for test cases that reference test cases in Set A; this constitutes Set B.
- (5) Analyse Set A and Set B for test step(s) reuse. The step(s) are identified as 'a', 'b', etc, and their frequency of occurrence is identified as '(x3)', '(x2)', etc.
- (6) Identify the dominant test step(s) in Set A test cases, i.e. those exhibiting a high frequency count. This constitutes Set C.

- (7) Identify which steps are common to Set B and Set C; this requires cross-referencing columns (4), (5) and (6) in the diagram. This gives a set of test cases which utilise the test steps in Set C; this set of test cases is Set D.
- (8) Convert Set D test cases into test scripts. Section 3.6 discusses an approach to this task. It should be noted that test cases which are not in Set D but appear in Set B form the basis for automating in the next cycle.

A core of scripts is now ready for execution against the next release of software. After execution, maintenance of these scripts may be necessary. If the required maintenance is low, due to a stable GUI and minimal execution problems, further conversion of test cases into scripts is possible. This will involve returning to the analysis performed earlier, to determine which set of test cases to select and convert from the remainder of the test specification.

This incremental approach produces a suite of scripts that evolve in harmony with the system under test.

The advantages offered by this scheme are that as the tests are automated in small sets it allows them to be ready for execution against the next release of system software. This, in turn, means they have the opportunity to mature early in the development cycle. Accelerating the maturity

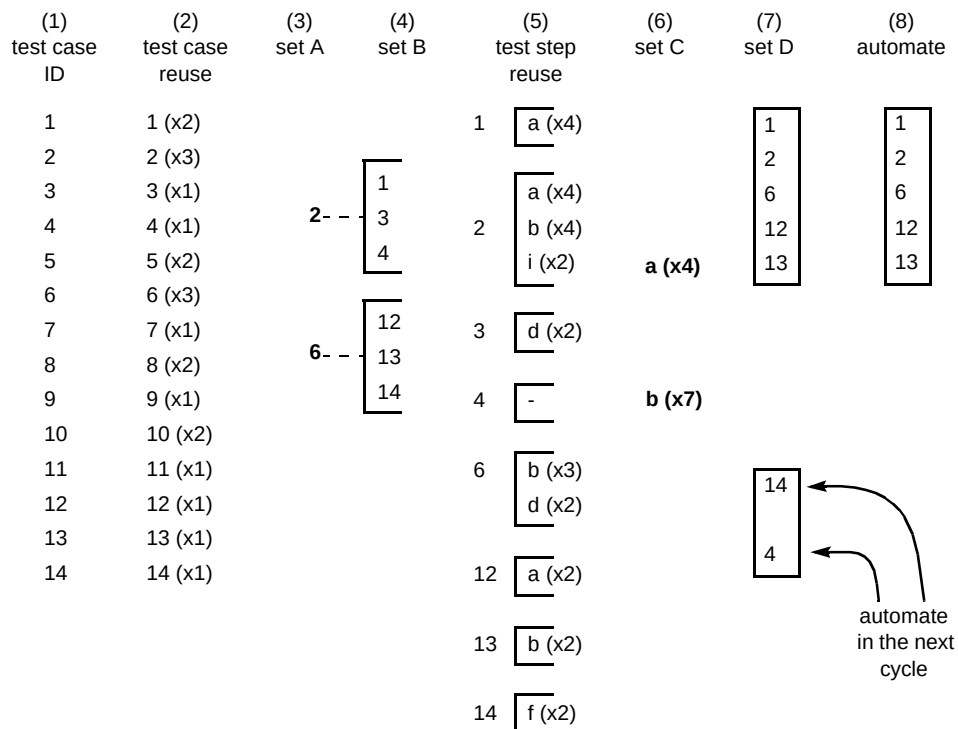


Fig 4 Test case selection.

increases their reliability over time and means they can be used with increased confidence in later test phases.

With this scheme we lessen the impact of subsequent GUI changes, as there is only a small core of scripts which will need modification. Using appropriate script design techniques can further reduce the maintenance overhead and this is discussed in section 3.6.

This coherent approach can be applied to any test specification adopting the IEEE829 standard and has the advantage that the person selecting the tests needs no particular knowledge of either the test specification or the system under test.

After test cases have been selected and grouped hierarchically, they may be further filtered using the complexity measures. The complexity of a test case is manifested in the type of attributes it displays. These attributes were discussed earlier (section 2). However, the major attribute required of a test case is to have a previous execution result of 'pass'.

This is not strictly a complexity measure but is required for a test to be successfully automated and is only applicable after the first test phase has completed.

The structured approach to test case selection allows a non-specialist to achieve the same results as a specialist. This de-skilling of the task creates more time for test script creation and other VV&T activities requiring more in-depth knowledge of the system under test.

This process of incrementally automating also addresses the need for maintaining the manual aspects of testing, as not all of the tests have been automated. This ensures an experimental approach is allowed to coexist with the automatic execution.

### 3.6 Test script design

Test cases can be converted into test 'scripts' by two processes when applied in the context of GUI software systems — capture/playback and manual coding.

Most, if not all, GUI-based test tools offer both processes but while the tool manufacturers extol the virtues of capture/playback functionality, there are important pitfalls to consider:

- synchronisation of the script with the application during playback,
- interaction of the script with asynchronous events,
- large dependency on stable user interface.

The synchronisation problem arises due to the fact that most applications do not respond in exactly the same way with each execution of the script. Positions of objects on the screen may vary from one execution to the next resulting in potentially erroneous script interpretations. Recorded scripts have no way to handle asynchronous events and may report inaccurate results. Experience of software system development provides the knowledge that most GUIs are unstable and hence changes in the user interface will require a significant amount of test script re-engineering in order to reuse the scripts in more than one regression activity.

With these pitfalls in mind the approach to script design is to programme the scripts by hand and to modularise wherever possible. Modularisation in this context is the process of defining commonly used actions and processes in a block of test code, known as a module. Within the module, the functional aspects can be enhanced by the use of conditional and iterative constructs. These methods build 'intelligence' into the module and make the scripts more robust to the environment in which they operate. The module can be considered a basic building block and scripts can be rapidly created by sequentially calling modules. Use of data files to store screen and system information ensures any modifications to the GUI can be reflected by minor changes to the appropriate file. The affects of these changes are global to all scripts which use them, thus reducing the amount of maintenance required. The following is an example of a section of test script built using this technique:

```
if (! call select_menu ("SCE3_main_W", "Profile",
"Open", "SCE3sel_list_W", "Left")) {
    report_msg ("ERROR Unable to locate Selection List
window");
    treturn (0);
}
```

This portion of test script shows the use of the "select\_menu" module. The parameters passed to it are, in the order shown:

- the short name for the main window,
- the menu category in this window,
- the menu option to select,
- the short name of the expected window to appear,
- the mouse button to use when selecting the option.

The short names are defined in data files initialised at the beginning of each script.

This call to "select\_menu" results in a sub-window being opened and recognised. If any of the windows do not appear, the module exits gracefully with a failure reason.

This methodology to script design overcomes the pitfalls listed earlier.

## 3.7 Making the most of the scripts

There are further elements of design that allow for the execution of scripts in batch mode. This mode of operation means executing the scripts sequentially without manual intervention and therefore allows the testers to manage their time to greater effect. The essential components required for batch processing are listed below.

- Modular test cases

Modular signifies a largely independent unit and in the context of a test case it means one which tests one element of application functionality. This approach tends to minimise the length of the test case and hence simplifies the automation task. It also allows sequencing of the scripts more readily as they can be categorised more succinctly.

- Start/stop points

Test cases should employ state '0' to state '0' structure, i.e. the application should, whenever possible, be left in the same state at the end of the test as it was at the start of the test. This aids in the creation of an automation execution schedule as each script is independent and reduces the set-up time when regression testing.

- Run plan

Executing the test cases in a specific ordering is less important if the state '0' approach to test case creation is adopted. However, the execution ordering of the test specifications is important as it helps in the high-level identification of appropriate test specifications to automate (i.e. test cases in one specification may require other test cases from another specification to be executed as a preliminary). Without this knowledge automation may be optimised in isolation.

## 3.8 Automation and standards

The test specifications are the natural foundation upon which to base an automation strategy. Emphasis on aspects of the design and layout of the specification provide the 'hooks' required by a strategy and expedite the acceptance of the tool into the existing working practices. The IEEE 829 standard [5] lays down a general test specification structure to which test cases should conform. The major elements of this are:

- test case identifier,
- test items,

- input specifications,
- output specifications.

The specific elements needed to aid an automation strategy might be described in the following groups:

- structural elements:
  - test case dependencies,
  - well-defined environment conditions,
  - formal references to other test cases or steps;
- metric elements:
  - number of test case dependencies,
  - total number of steps (precondition + input),
  - number of test cases referenced.

The structural elements ensure the test case is defined comprehensively. This removes any ambiguity or vagueness that is a significant barrier when converting a test case into a script. The metric elements aid the selection process and provide an input to identifying complexity.

These elements reflect the design principles of script creation and execution while also allowing the selection process to be implemented with increased efficiency.

## 3.9 Summary

Many previous approaches to test automation within a GUI environment have concentrated on selecting the right tool for the job with little or no consideration of its application once purchased. Within large integration projects this naive approach has not produced the expected results. Initially focusing energies on analysis and measurement of the test specifications gives a clear and informed view of where to direct the test effort most effectively. Bringing together the methods for script design and infrastructure ensures the integration of automation into the existing working environment.

## 4. Application of test automation in SI

Many projects within BT have made use of test automation tools to reduce testing costs and time-scales. The experience of two projects, Work Manager (WM) and MCSS for Cambridge are briefly described here.

### 4.1 Work Manager

The test suite owned by the WM systems integration and test (SI&T) team is divided into two areas.

- User interface transaction tests

The automated testing of user interface transactions use a tool called MINT. The MINT tool emulates the user keyboard actions and captures the display characters. The tool is driven from scripts that have a C type construct and the capability to be modularised; thus standard libraries can support many test users executing the same validation functions. Data retrieved is used in comparisons with baselined data files and script variables. This tool is used for automating tests of user interfaces that are character based and run on Unix and DOS platforms.

- Auto-processing tests

The auto-processing tests cover all the WM background process testing; hence the test activities are focused more at the lower system level. Test automation at this level makes use of a tool called Autotest, which interprets user-defined keywords to a set of system calls. These can then be used to drive the test input, apply the test stimuli, gather the output and compare it to the expected output.

During WM V6.0 System Test, there were 6420 user-interface test cases executed, of which approximately 95% were automated. For the same release, there were 6506 auto-processing test cases, of which 47% were automated.

The use of test automation in Work Manager has provided several benefits. The most significant is in reducing manpower effort and preventing slippage of delivery time-scales. Table 5 illustrates this for the three WM sub-systems discussed in section 2.

Table 5 Savings for three WM sub-systems.

|                                | SS1  | SS2  | SS3   |
|--------------------------------|------|------|-------|
| % Test automation              | 99%  | 100% | 46%   |
| Manual effort total (days)     | 20.5 | 27   | 18.28 |
| Automation effort total (days) | 4    | 9.6  | 17.27 |
| Manpower saving                | 16.5 | 17.4 | 1.01  |

The testing manpower effort is compared directly with the previous manual test effort. The percentage automation indicated how many of the tests for the sub-system were automated, and the manual value is derived from this percentage of the test set.

This manpower cost saving has been particularly beneficial to enable the WM SI&T team to prevent extensions to time-scales, while the WM product complexity increases between releases. It has been noted that team members who own test areas which are largely automated, are invariably able to either complete on time (particularly where their test area has undergone significant functional change) or earlier than planned.

Within WM, the test automation approach has always been 100% (i.e. all test input, test stimuli, test output and test results gathering is automated). Where, for example, the test automation benefits were only in entering test input-data, this approach is counter-productive. A multi-level test automation approach is needed, such as shown in Table 6.

Table 6 Example multi-level test automation approach.

| Level | Automation coverage              |
|-------|----------------------------------|
| 1     | Automated Test Input             |
| 2     | Level 1 + Automated Test Stimuli |
| 3     | Level 2 + Automated Test Output  |
| 4     | Level 3 + Automated Test Result  |

While work in this area is still under investigation, early indications have shown that this approach can better support a wider variety of test automation requirements.

#### 4.2 MCSS — telemarketing services

Test automation was used during system testing of MCSS for 'BT Telemarketing Services' major delivery 4 (MD4). MD4 provides customers with a GUI through which they can manage their telemarketing service and receive electronic reports on calls delivered to their numbers.

##### 4.2.1 Rationale

The use of automation on MD4 testing had a number of primary drivers:

- reduce the cost of testing,
- improve the quality and repeatability of the tests,
- test the non-functional requirements.

Test automation was perceived to reduce the overall development cost of MD4 and of subsequent releases of MCSS deliveries to telemarketing services.

There was the need to increase the quality of the testing by improving test accuracy, repeatability and coverage. This was to be achieved through automating the labour-intensive and manual 'button-pushing' tasks of testing the functionality of the GUI.

The stress, performance and capacity testing could only be performed using test automation. It was therefore inevitable that automated testing would play an important role during the system testing of MD4.



#### 4.2.2 Coverage

Automated tests were applied in the following five domains.

- Test precondition data

Each test cycle required the set-up of at least 160 customers, each with at least one user. Test scripts were used to perform the set-up while at the same time test the user administration functionality and the security features built into the user interface.

- Functional testing

Of the four MD4 functional areas, the 'reports' functionality was most amenable to automation, and as such was automated in its entirety. The 'administration testing' functionality was partially automated. There was, however, little or no automation of the 'call plan editing' or the 'near-real-time update' functionality.

- Stress test functionality

While near-real-time update (NRTU) functional testing could not be automated, the stress testing could. Test scripts were used to continuously apply NRTUs to plans throughout the test cycle. This resulted in over 4000 NRTUs being executed in a test cycle.

- Capacity testing

In order to validate that the system resources could support the forecast customer demands, the customer data was loaded on to the system with the forecast capacity for each cycle. This required loading at least 200 customers, 5000 call plans and 14 days of reports for all the customers.

- Load and performance testing

Automated testing was used to obtain performance characteristics of the system under load. The testing was able to emulate the log-on profile of users and to simulate their activities. With the test tools it was possible to test 180 concurrent users on the system and to obtain the characteristics of the system under various load conditions.

#### 4.2.3 Process

Automated testing was performed using two main tools, WinRunner V 3.0 [9] and PurePerformix [10]. WinRunner was used to test the functions of the GUI from an end-user perspective. Most of the test scripts were developed by tailoring a set of base functions to specific test cases. These functions were then linked in a test driver. The base functions were manually coded by specialist expertise and the tailoring was performed by the tester leading the functional area to which the tests were targeted. WinRunner

was also used to stress and capacity test MD4. The scripts for these activities were developed by capturing activity on the user interface with the test script subsequently being played back in a repetitive loop throughout the test cycle.

PurePerformix was used to test the load and performance characteristics of the server. The test scripts were developed using a combination of tool capture/playback functionality and manual coding techniques. The scripts were then compiled and played back under the control of a test harness, which also provided the capability to simulate realistic load ramp-up on the system.

#### 4.2.4 Benefits

The use of automation on MD4 has resulted in several benefits. While cost is one of these benefits, the automation contributed to reducing the test schedule and increasing the overall product quality.

In Table 7, the automated testing effort is directly compared to manual effort previously required to test the same functionality. The table does not include the costs of developing the test scripts. Nonetheless, the manpower savings per test cycle have been spectacular for the 'reports' functionality where the testing of the application is virtually all automated.

Table 7 Savings per test cycle for MD4.

|                                | Reports | Admin |
|--------------------------------|---------|-------|
| % Test automation              | 99%     | 10%   |
| Manual effort total (days)     | 30      | 5     |
| Automation effort total (days) | 3       | 4     |
| Manpower saving                | 27      | 1     |

In contrast, this level of saving was not achievable with the 'administration' functionality where the user interaction with the system is not as structured as reports.

The use of automation enabled quantitative evaluation of performance bottle-necks in the system. The results of the evaluation were used to focus the development effort on areas that yielded the most benefit or where impact to the users was greatest.

#### 4.2.5 Challenges

While the automation did prove beneficial, there were many challenges, some of which were:

- automation of unsuitable test cases,
- migrating manual tests to automated scripts,
- GUI instability.

The testing of the 'call plan editing' functionality required the use of base functions that were hand-crafted by a tool specialist. However, using these base functions proved to be difficult because the testers had to describe the graphical layout of the call plan using function calls. This was a slow and tedious process and was eventually abandoned as it was not amenable to automating and was costing more than the equivalent manual testing.

Automated testing was not considered from day one of the test regime, because MD4 was initially planned as a single release that equated to a single test period of twelve weeks. However, subsequent releases of MD4 were delivered, which extended the test period. The consequence of not considering automated testing earlier in the test planning and design stages was that the test cases had to be modified before conversion into scripts.

During test automation, the user interface was unstable. It was therefore necessary to rewrite some of the scripts and recapture the validation views. This negated some of the benefits of automating but is an expected consequence of automating in a GUI environment.

#### 4.2.6 Lessons

A number of significant lessons were learned during the testing of MD4, namely:

- automating appropriate parts of the testing yields cost and quality benefits,
- automation must be considered at the early stages of test planning,
- automating in a GUI environment requires particular consideration.

The issues noted above have been developed in sections 2 and 3 and future test automation work in MCSS will ensure these aspects are addressed.

Overall, the experience of automating MD4 testing has been a positive one with significant productivity gains.

#### 4.3 Analysis

Table 8 provides a comparison of the level of automation between WM and MCSS system test phases.

Table 8 Percentage test automation in recent releases of WM and MCSS projects.

|                 | WM V6.0 | MCSS MD4 |
|-----------------|---------|----------|
| Total tests     | 12923   | 523      |
| Automated tests | 9212    | 239      |
| % Automation    | 71%     | 46%      |

The comparison indicates that it has not been possible to completely automate the testing task. The manual aspects of testing may never be completely replaced by automation and neither should they be, for two main reasons:

- preservation of the experimental aspects of testing is required,
- not every type of test is amenable to automating.

These two points have been discussed at length during the first three sections of the paper.

#### 5. Conclusions

Test automation has become an integral and necessary aspect of systems integration activities. The reliance on test tools is now at a stage where their usage is crucial in expediting product and service delivery while maintaining comprehensive test coverage.

The work presented in this paper has discussed test automation methodologies and techniques and the results of their application in two systems integration projects have been presented.

As integration projects continue to increase in scale, it is now clear that three main factors need to be addressed if automation is to provide the optimum benefits:

- integration of test tools into VV&T processes,
- suitable analysis of test specifications as a starting point to test automation,
- minimising the risks of GUI instability via an iterative automation approach.

These principles have been discussed in this paper, specifically from the perspective of character and GUI software systems integration. Only two examples were given, that of Work Manager and MCSS, but it is believed that these principles are generally applicable to large systems integration activities and in many cases are actively being employed.

BT's Systems Integration department has demonstrated, in many projects, that test automation plays a major role in timely delivery of products and services. The department will continue to ensure the most appropriate tools and practices are utilised in future integration activities.

## Acknowledgements

Thanks to Mike Saucedo and Mike Sapsed for constructive feedback and guidance.

## References

- 1 Myers G J: 'The art of software testing', Wiley, New York (1979).
- 2 Norris M and Rigby P: 'Software engineering explained', p 106, Wiley (1992).
- 3 South G J: 'Automating complex systems', MSc Dissertation (1994).
- 4 Hicks I D: 'Automation of GUI software systems', MSc Dissertation (1995).
- 5 IEEE Standard for Software Test Documentation, ANSI/IEEE Std 829 (1983).
- 6 Kepple L R: 'The black art of GUI testing', Dr Dobb's Journal (1994).
- 7 Graham D R: 'The CAST report', Grove Consultants, Unicom Seminars Ltd (1991).
- 8 Beizer B: 'Software Testing Techniques', Second Edition, Van Nostrand Reinhold, New York, p 459 (1990).
- 9 WinRunner User Guide, Mercury Interactive.
- 10 PurePerformix User Guide.



Ian Hicks joined BT as an apprentice in 1980. He received a BEng Hon (Telecommunications) degree from the University of Essex in 1991 and a BT MSc in Telecommunications from University College, London and BT jointly, in 1996. Since he graduated in 1991, Ian has been involved with the IVV&T of various large projects, including CVNS service management, service creation environment and most recently number portability where he is responsible for end-to-end IVV&T.



Graham South joined BT as an apprentice in 1981. He completed a BEng in Electronics at Reading University in 1989 and the BT MSc in Telecommunications in 1996. He joined the NOMS2 VV&T team in Ipswich in 1989 and has since been involved in Work Manager where he leads a team responsible for the integration of the WMS product. He also chairs a work area Test Automation forum, which has a current special interest in test management.



Services Integration Team.

Abimbola Oshisanwo joined BT in 1989 on completion of a PhD in Electrical Engineering from the University of Waterloo in Canada. Upon joining BT he contributed to research to define an object oriented architecture for the software of telecommunications services. He also worked on service creation environments for Intelligent Networks. He joined the Systems Integration department in 1994, and has since managed a number of projects that integrate MCSS for BT Telemarketing Services with the backend network systems. He currently leads the MCSS for BT Telemarketing